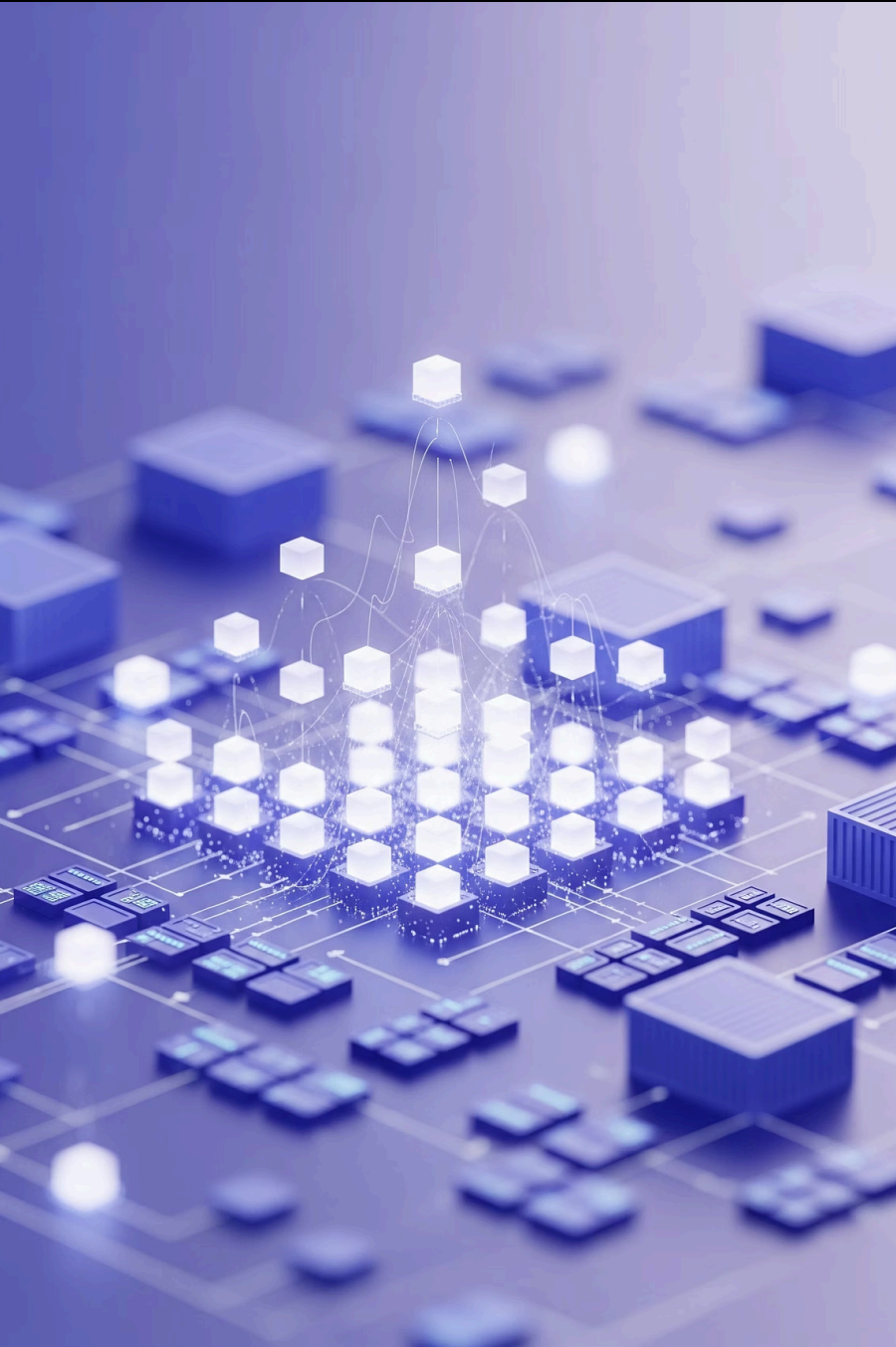




Scaling Model Serving with Kubernetes

Dynamic scaling for reliable, efficient inference in production environments

Why Kubernetes for Model Serving?

Elastic Scaling

Dynamically handles varying inference workloads based on demand patterns

Infrastructure Abstraction

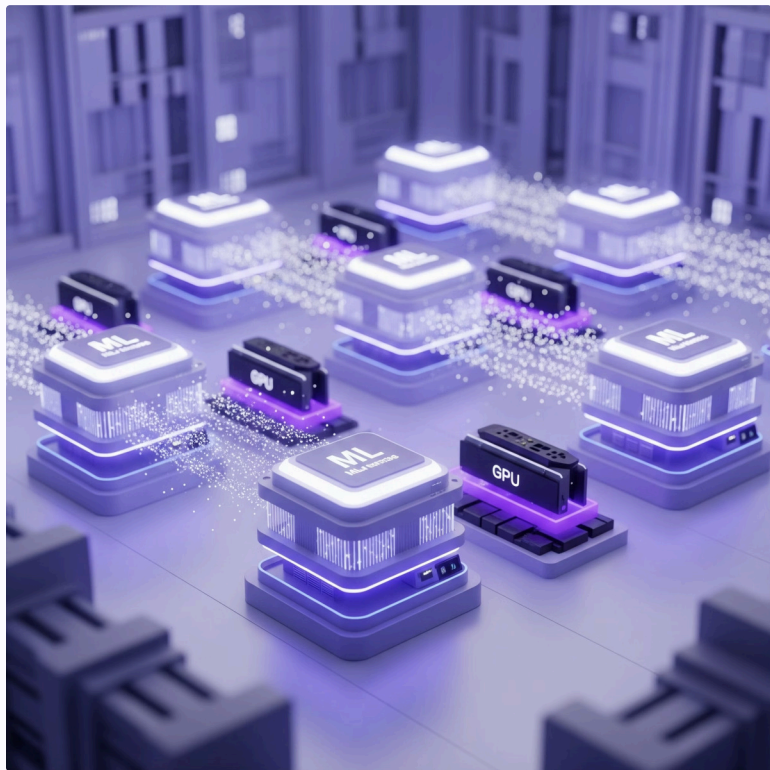
Deploy consistently across cloud providers and on-premises environments

Resource Integration

Native support for GPU scheduling, autoscalers, and custom operators

Multi-Tenant Architecture

Enables secure deployment of multiple models with isolated resources



Core Building Blocks



Deployment

Manages pod lifecycle, replica count, and rolling updates for model servers



Service

Provides stable network endpoints and load balancing across model replicas



HPA

Horizontal Pod Autoscaler adjusts replica count based on metrics like CPU/GPU usage



Ingress / Gateway API

Layer 7 routing, traffic management, and TLS termination for inference endpoints



ConfigMaps/Secrets

Separate configuration from code; securely manage model credentials and settings

Autoscaling Options

Horizontal Pod Autoscaler (HPA)

- 1 Scales number of inference pods based on CPU/GPU utilization or custom metrics
Example: `kubectl autoscale deployment inference-api --cpu-percent=70 --min=3 --max=10`

Vertical Pod Autoscaler (VPA)

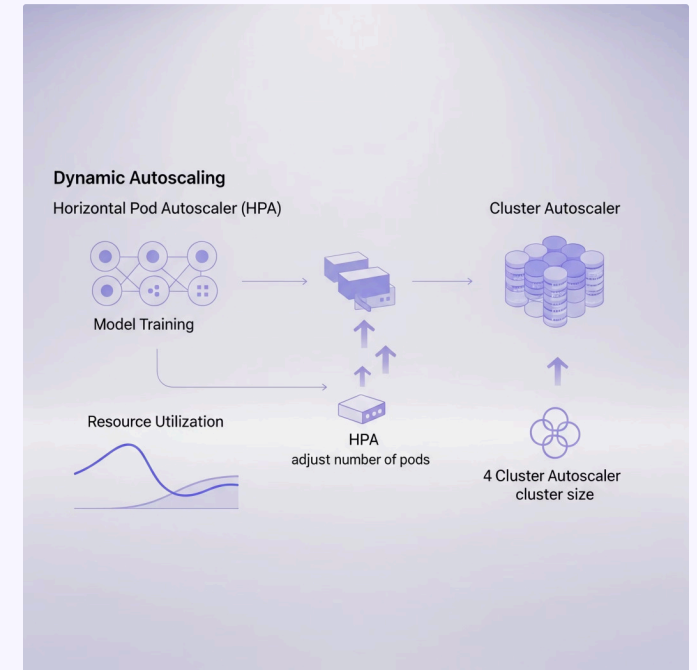
- 2 Dynamically adjusts CPU/memory resources allocated to containers
Ideal for gradually optimizing resource allocation based on historical usage

Cluster Autoscaler

- 3 Automatically adds or removes worker nodes when pods can't be scheduled
Works with cloud provider node groups and machine deployments

Custom Metric Scaling

- 4 Scale on business metrics: queries per second, inference latency, queue depth
Requires metrics adapter to expose Prometheus/custom metrics to K8s API



Layered autoscaling approach provides flexibility for ML workloads



GPU Scheduling & Placement

Resource Allocation

Request GPUs explicitly in pod specs:

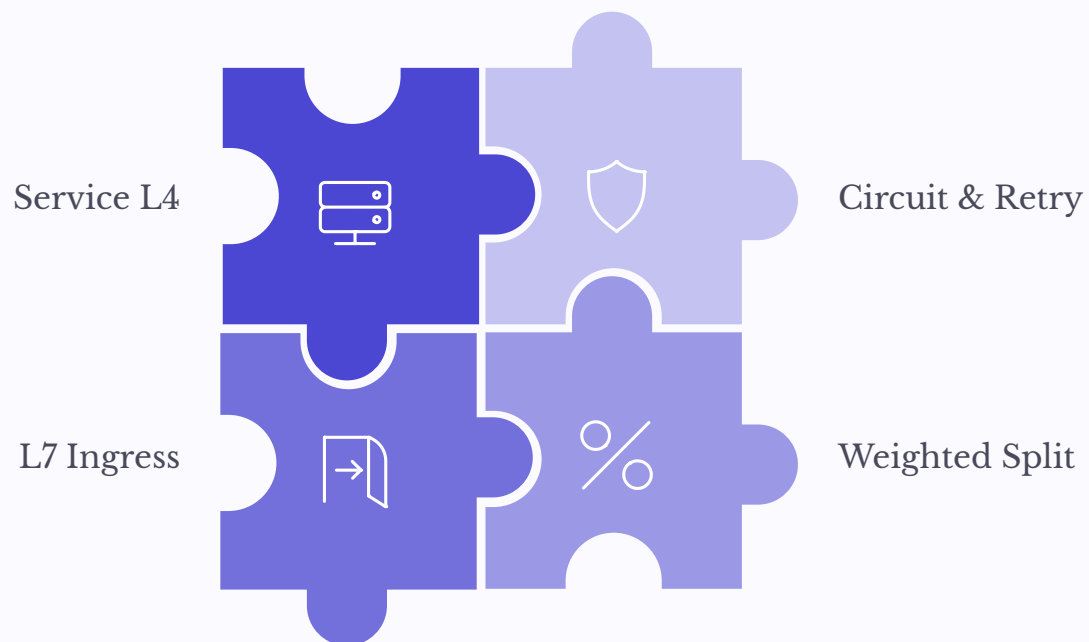
```
resources:  
  limits:  
    nvidia.com/gpu: 1  
  requests:  
    nvidia.com/gpu: 1
```

GPUs are non-compressible, whole-number resources

Advanced Placement

- **Pod affinity/anti-affinity:** Control locality for GPU access and high availability
- **Node selectors / taints:** Reserve GPU nodes for inference workloads only
- **MIG support:** Partition NVIDIA A100/H100 GPUs into smaller slices for efficient sharing

Load Balancing Patterns



L4 Service Distribution

Kubernetes Services provide TCP/UDP load balancing across model replicas

Traffic Splitting

Weighted routing enables canary releases, A/B testing, and gradual deployment of new models

L7 Application Routing

Ingress controllers (NGINX, Envoy) route by path, headers, or model version

Resilience Patterns

Circuit breaking, outlier detection, and automatic retries improve reliability

Multi-Model Hosting

Deployment Strategies

Dedicated Pods

One model per pod set

Pros: Isolation, independent scaling

Cons: Resource overhead, more complex management

Multi-Model Server

Multiple models in one server (Triton, TorchServe)

Pros: Resource sharing, simplified management

Cons: Potential interference, complex scaling

Model Deployment Workflow

1. Store models in versioned registry (MLflow, Weights & Biases)
2. Deploy with CI/CD pipeline (GitOps, ArgoCD)
3. Use canary or shadow deployments to validate models
4. Monitor performance metrics during rollout

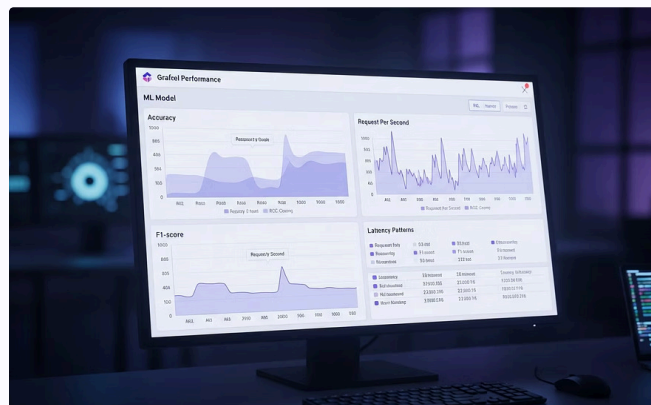


Observability & SLOs



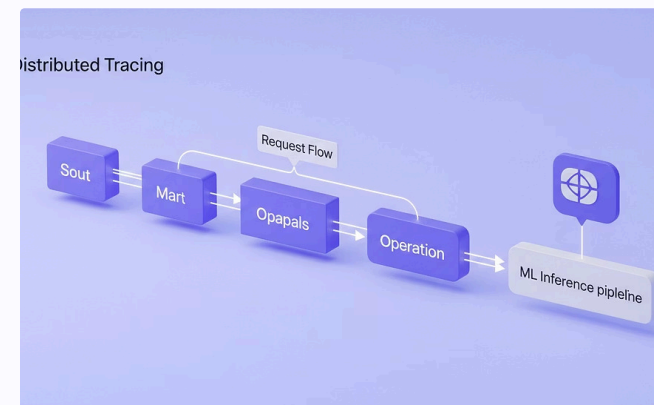
Metrics Collection

- Prometheus metrics for QPS, latency (p95/p99), error rates
- GPU utilization, memory usage, queue depths
- Export metrics from model servers with exporters



Visualization

- Grafana dashboards for inference pipelines
- Model-specific KPIs and health indicators
- Alert thresholds based on SLOs



Tracing & Debugging

- OpenTelemetry for request tracing across services
- Track requests from LB through preprocessing, inference, post-processing
- Correlation IDs for debugging complex failures

Reliability & High Availability

Multi-Zone Distribution

Deploy replicas across availability zones to survive zone failures:

```
podAntiAffinity:  
  requiredDuringSchedulingIgnoredDuringExecution:  
    topologyKey: "topology.kubernetes.io/zone"
```

Controlled Maintenance

Pod Disruption Budgets ensure minimum availability during upgrades:

```
minAvailable: 2 # or  
maxUnavailable: 25%
```

Health Checks

Readiness/liveness probes prevent routing to unhealthy pods:

- Readiness: Is the model loaded and ready to serve?
- Liveness: Is the container functioning properly?
- Startup: Allow time for large models to load

Chaos Testing

Validate resilience by simulating failures:

- Pod terminations, node failures, network partition
- Resource exhaustion and latency injection
- Use tools like Chaos Mesh or Litmus

Design for failures: they will happen in production!

Real Time

Vernal Time

GPU / Info

Active GPU Nodes

Slurde

ML Intage

Slurde

Spammedia

Purds

Mistatrole

Ialstage

GPU Utilization

1,000

900

800

700

600

500

400

300

200

100

0

ML Invald time

ML Invald 8%

180

170

160

150

140

130

120

110

100

90

80

70

60

50

40

30

20

10

0

ML Invald time

ML Invald 8%

Current Spending on GPU GPU Resources

30.90%

13.90%

90.20%

2.60%

0.29%

85.40%

10L Project

ML Project

Bue Team

UML Team

Dice Mant

MPMer Treard

Total Available GPU

GPU Utilized: 33%

Total Available: 20%

Yct. Mant

772.60%

772.60%

Mammosha Active GPU Nodes

576.30%

Prossure Coudance

46.98%

Mapt Scaport

Jobs Available on

10.45%

Thousd GPU Memory

772.60%

Thousd GPU Memory

Jobs Currently GPU

Quance Houdance

Thousd Houdance

1 Right-Size Resources

Avoid over-provisioning GPUs by:

- Batch inference for higher throughput
- Monitor actual utilization patterns
- Use smaller GPU types where possible

2 Leverage Spot/Preemptible Instances

Save 60-90% with ephemeral compute:

- Implement checkpointing for stateful workloads
- Use PodDisruptionBudgets and tolerations
- Mix spot and on-demand for reliability

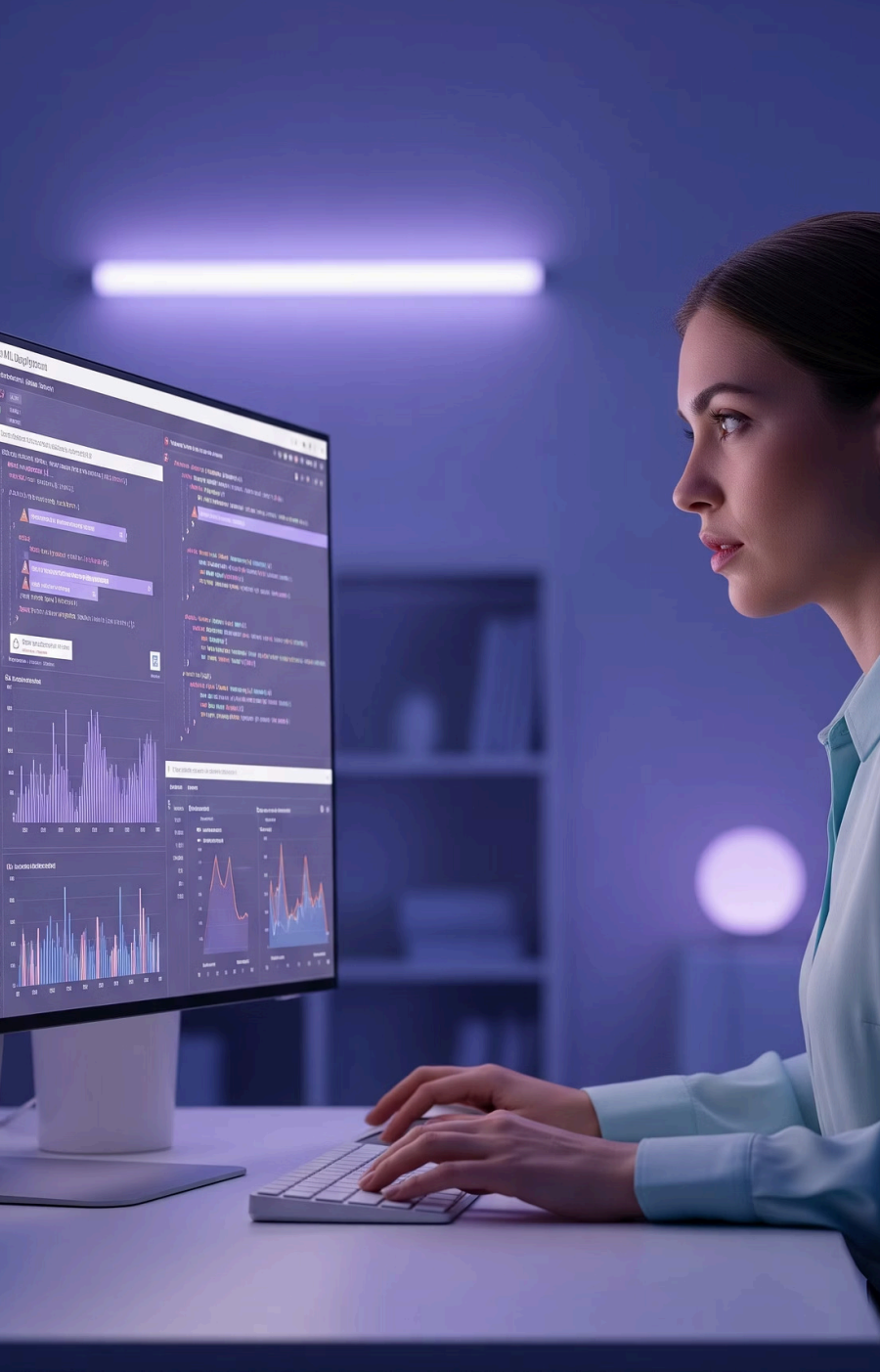
3 Efficient Autoscaling

Scale to zero for intermittent workloads:

- KEDA for event-driven scaling
- Balance cold start latency vs. cost

4 Separate Training & Inference

Different workload characteristics require different infrastructure approaches



Common Pitfalls

Resource Misconfiguration

Not setting proper resource limits leads to "noisy neighbor" problems

```
# Missing memory limits can cause OOM kills
resources:
  requests:
    memory: "4Gi"
# Missing limits!
```

Incomplete Scaling Metrics

Only scaling on CPU/GPU utilization ignores actual workload characteristics

Better approach: Include QPS, latency, and queue depth metrics

Poor GPU Utilization

Deploying single model per GPU node often wastes expensive resources

Solution: Use MIG, multi-model serving, or smaller GPUs for right-sizing

Cold Start Latency Surprises

Aggressive autoscaling can cause latency spikes during scaling events

Mitigation: Implement prewarming, keep min replicas, or use predictive scaling

Best Practices & Takeaways



Scalable Infrastructure

Kubernetes provides the backbone for flexible, resilient model serving across environments



Multi-Layer Scaling

Combine HPA + Cluster Autoscaler + custom metrics for true elasticity that handles traffic spikes



SLO-Driven Operations

Implement comprehensive observability and use SLOs to drive scaling and reliability decisions



Separation of Concerns

Inference infrastructure requires different optimization strategies than training infrastructure

❏ Next Steps

- Evaluate current inference bottlenecks
- Implement appropriate autoscalers
- Set up observability pipeline
- Run chaos tests to validate resilience